World Models in Action

2. The anatomy of a world model

This chapter covers

- What a state has to represent, and hold onto over time, before it quietly fails
- Why a dynamics model has to predict the right counterfactual, not just what's likely
- How reward, value, and search hide real design decisions
- What actually separates a world model from a sequence model, a generator, or a simulator
- A test for telling a useful world model from one that only looks the part

In Chapter 1, you saw a simple working loop consisting of a state built by an encoder, a dynamics model to predict the possible changes in that state, and a planner/policy to use those predictions to perform an action. Now let's discuss what's actually inside each of them. What exactly should a state be made up of to survive an environment it has never seen before? What things a dynamics model has to get right before it becomes trustworthy? What should an objective look like so a planner can use it well? What keeps this whole thing coherent over time.

By the end of this chapter, you'll have a way to look at some new architecture, a video predictor, a diffusion model, a policy network someone is calling a world model and you'll be able to tell whether it earns the name or just looks like one. You'll be able to reason about the design principles behind these parts.

A world model can be viewed as four interacting parts enclosed by persistent state across time. The state or belief summarizes what the agent currently needs to know; the dynamics model predicts how that state could evolve under different candidate actions, branching into imagined futures; the objective scores those futures against the agent's goal; and action selection commits to one action. That action changes the real world, producing a new observation that updates the persistent state for the next tick. The test at the bottom evaluates the complete system through prediction fidelity, long-horizon consistency, action grounding, and planning utility.

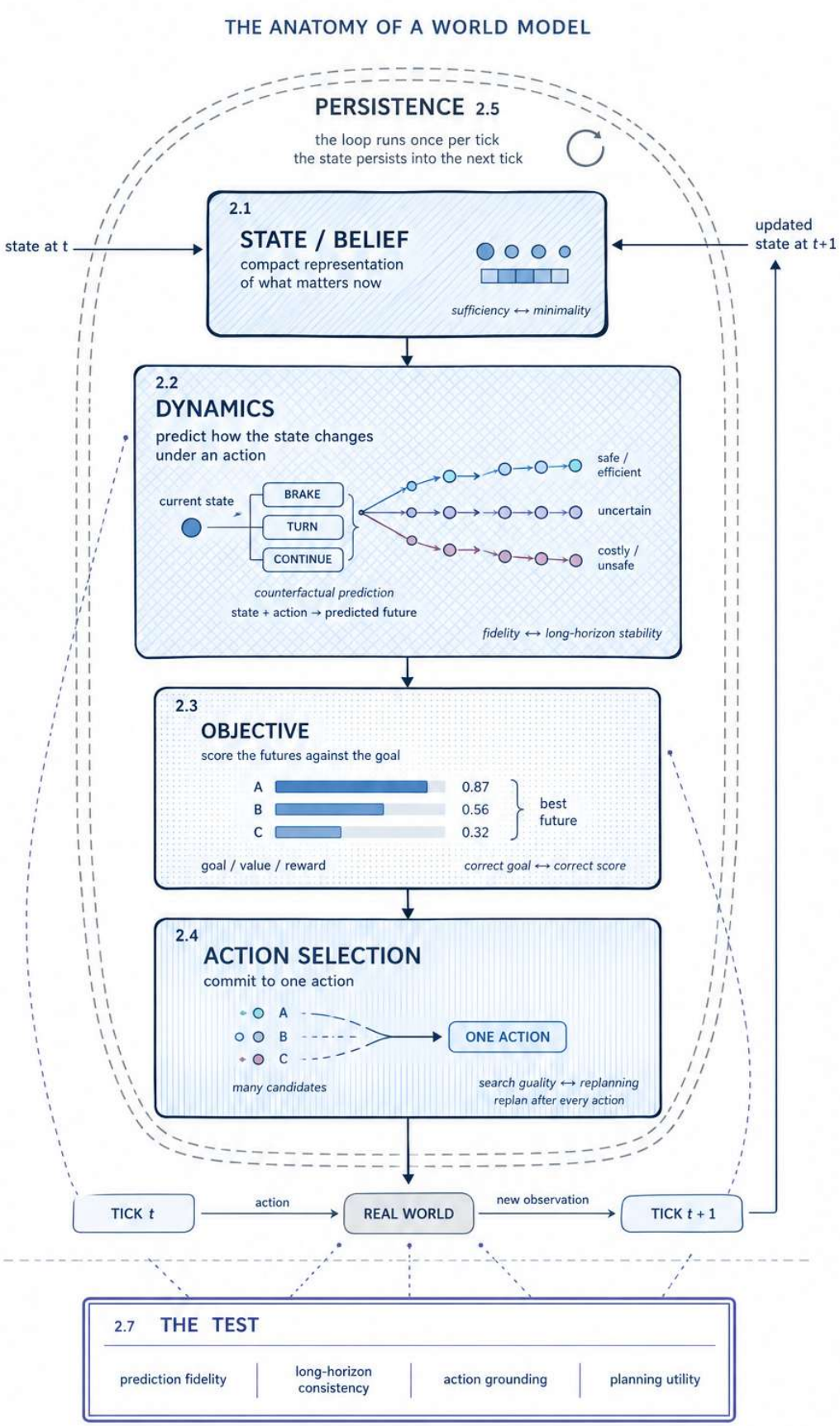

THE ANATOMY OF A WORLD MODEL
PERSISTENCE 2.5
the loop runs once per tick
the state persists into the next tick
state at t
updated state at t+1
2.1
STATE / BELIEF
compact representation of what matters now
sufficiency ↔ minimality
2.2
DYNAMICS
predict how the state changes under an action
current state
BRAKE
TURN
CONTINUE
safe / efficient
uncertain
costly / unsafe
counterfactual prediction
state + action → predicted future
fidelity ↔ long-horizon stability
2.3
OBJECTIVE
score the futures against the goal
A 0.87
B 0.56
C 0.32
best future
goal / value / reward
correct goal ↔ correct score
2.4
ACTION SELECTION
commit to one action
A
B
C
ONE ACTION
many candidates
search guality ↔ replanning
replan after every action
TICK t
action
REAL WORLD
new observation
TICK t + 1
2.7 THE TEST
prediction fidelity
long-horizon consistency
action grounding
planning utility

Figure 2.1 The anatomy of a world model, at a glance.

2.1. Observations, state, and belief

The understanding of how the world works is limited by the complexity of perception some observer has. That's true for machines and humans alike. An encoder can't see the 'actual' state of the warehouse. It usually just sees a camera frame, a depth reading, maybe some other sensors, and has to build a 'belief' out of that evidence. For most of our use cases, the belief and the truth are so close that the distinction feels purely academic. Its importance becomes clear when the two come apart, which happens constantly in an environment that isn't fully visible from wherever the sensors are pointed.

> **NOTE** An environment where the current observation doesn't tell you everything you'd need to know is called partially observable in the reinforcement-learning literature, often shortened to POMDP. It's the normal condition of almost every real environment, and it's exactly why a state has to be built from history rather than read off the current instant.

The state has to preserve whatever from the past will matter for the future. In other words, a state is sufficient if nothing in the observation history helps predict the future better than it already does. So, if we hand over to one person the entire history of the camera frames and sensor readings, and hand another person just the current state vector, and if neither can out-predict the other, we say that the state is sufficient. Everything extra the first person has is redundant. This is the type of compression we want in our system.

Well, that's the ideal case, which we called Markov: a state so complete that the rest of history adds nothing further, for any purpose it may be used downstream. Getting a trained encoder to reach that is difficult, and not even needed. It just has to be sufficient for the task and horizon we actually plan over, not for 'every question' downstream. That target is lower and the one we try to get.

A state then has to carry information across time, not just compress what's visible right now. That would be just a snapshot. A state answers what we know, given everything we've seen so far. A snapshot is discarded the instant the next one replaces it. A state is supposed to keep what still matters, and that property is what aliasing and minimality depend on.

The easy version of a problem we face is that a parked forklift and a passing forklift can look very identical, so a frame-only state is not able to tell them apart and predicts a blurry average of those futures. The actual version we face is say the forklift is visible for two seconds, then disappears behind an opaque shelf for a while, and then comes back into our view. If the state only summarised what's currently visible, it'll alias the moment the forklift hides behind the shelf, exactly the same failure as the single frame. It'll just stretch it over a few seconds instead of one instant. A history-based state is smarter in the sense that it can keep believing the forklift is there, predicting its trajectory roughly, and only update that belief when a new observation confirms or contradicts it.

Well, that requires more than just seeing factual history though. It requires the system to carry a conclusion, or a wondering, forward through moments where the new observations say nothing at all about that conclusion. An observation of the empty floor where the forklift was last seen is an absence of evidence, not proof of forklift's vanishing, and for a state to treat this absence of evidence as the evidence of absence is to alias just as badly as the frame-only, with an even worse timing. This is actually a memory problem rather than an encoding problem, and we'll study about that in section 2.5. For now, aliasing happens by default rather than just a one-time problem, whenever we have two different situations compressing into representations too close to tell apart. That could be a result of a frame missing detail, short memory, or a belief being erased when nothing is actively confirming it.

Belief state

A belief state is the encoder's running estimate of what's actually going on in the environment, built from everything observed so far rather than the current observation alone. It's a task-relevant summary inferred from observations and history, not a literal reconstruction of the physical state, and it only has to capture what prediction and action actually need, not everything that's true about the environment. When people say "state" casually, they sometimes mean this belief state and sometimes mean the true underlying situation. This book almost always means the belief state, since the true state usually isn't available to the agent at all.

We say that for an encoder, smaller is cleaner, cleaner is better. This minimality is an engineering budget, and a sweet spot is what we want. A latent dimension too small gives us aliasing, forcing different situations to share the same handful of numbers because there's no

room. Making it too large will get us a quieter cost rather than a guaranteed failure. The dynamics model would need to learn more as it has to model the evolution of a bigger space, which might include irrelevant things like the motion of leaves. Minimality is getting as small as sufficiency allows, and no smaller, and that's what we'll discuss in chapter 3.

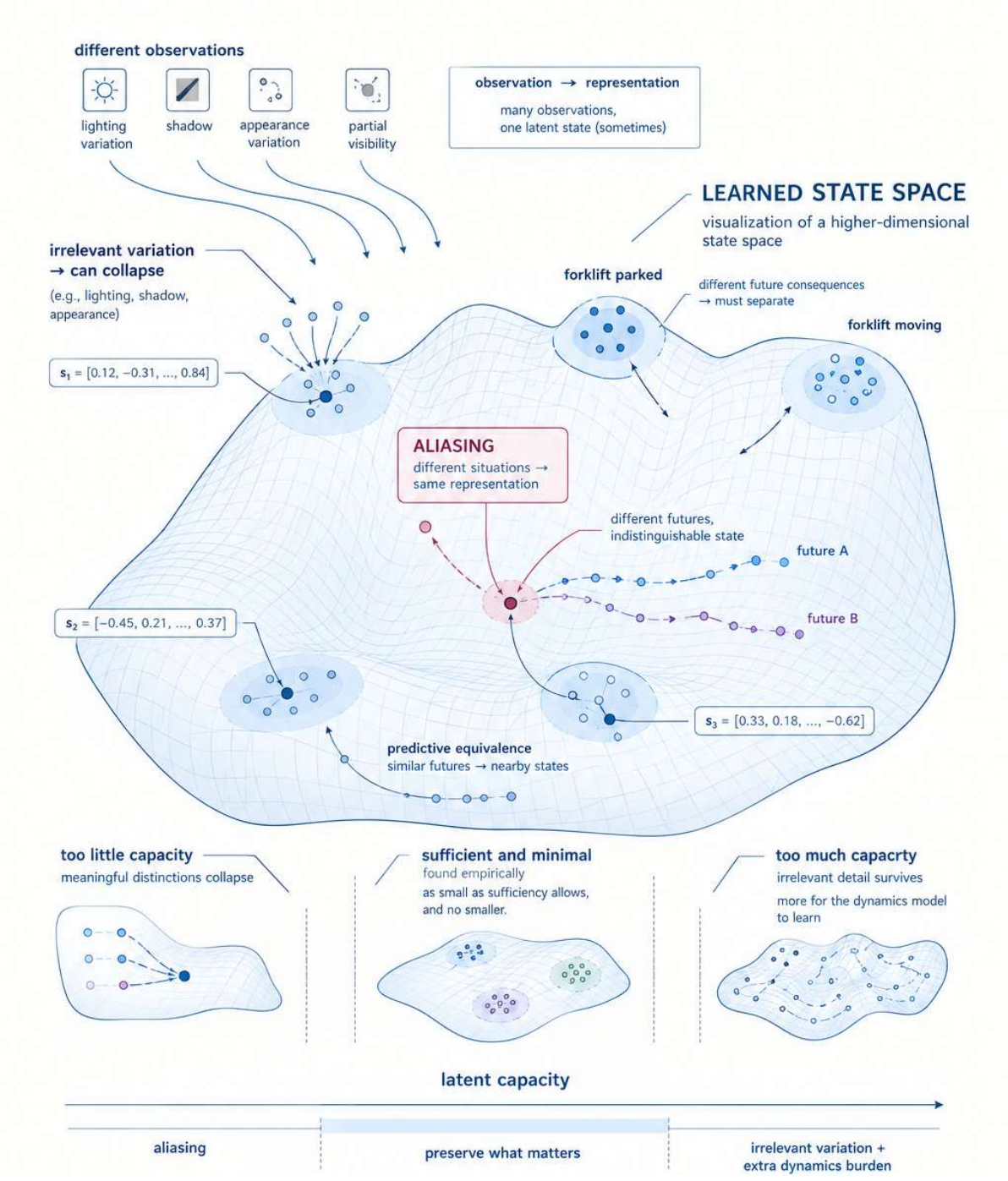


Figure 2.2 The geometry of a useful state representation.

Different observations can map to nearby latent states when their differences are irrelevant to prediction or action. A useful state instead keeps situations with different future consequences distinguishable. The aliasing example shows two different situations collapsing into an indistinguishable representation, causing their different futures to become ambiguous to the dynamics model. The lower capacity spectrum illustrates the trade-off: too little latent capacity causes meaningful distinctions to collapse, while too much preserves unnecessary detail and increases the burden on the dynamics model. The desired representation is therefore the smallest one that remains sufficient for the task and planning horizon, with that useful region determined empirically.

Older robotics work used to hand-design the state. Variables like position, velocity, heading and distance to obstacle were entered by a person who decided in advance which numbers mattered. That works as long as we know for sure which variables are relevant and every sensor maps cleanly onto one of them. It falls apart the moment the input is a camera feed instead of a handful of clean sensor readings, because nobody can hand-write a formula that turns pixels into "is that a forklift or a shadow." Learning the encoder instead, which is what chapters 3 and 4 spend their time on, hands the sufficiency and minimality trade-off to gradient descent on real data, instead of to a person guessing in advance which variables to keep.

Here’s the smallest possible way to see the aliasing failure in code, not because you’d ever ship either of these, but because it makes the problem concrete instead of abstract (figure 2.3).

PYTHON

```python
# a frame-only encoder: no memory of anything before this instant
def encode_frame_only(obs):
    return featurize(obs)              # forklift missing -> state says "no
forklift"

# a history-aware encoder: carries a belief forward
def encode_with_memory(state, obs):
    if obs.forklift_visible:
        state.forklift = update(state.forklift, obs)   # new evidence, update
the belief
    else:
        state.forklift = decay(state.forklift)          # no evidence either
way, don't erase it
    return state
```

The first version aliases the moment the forklift leaves frame, because it has nothing to fall back on except what it can currently see. The second at least has the option to hold a belief across the gap. Whether it holds the right belief for the right amount of time is a much harder question, one section 2.5 comes back to properly.

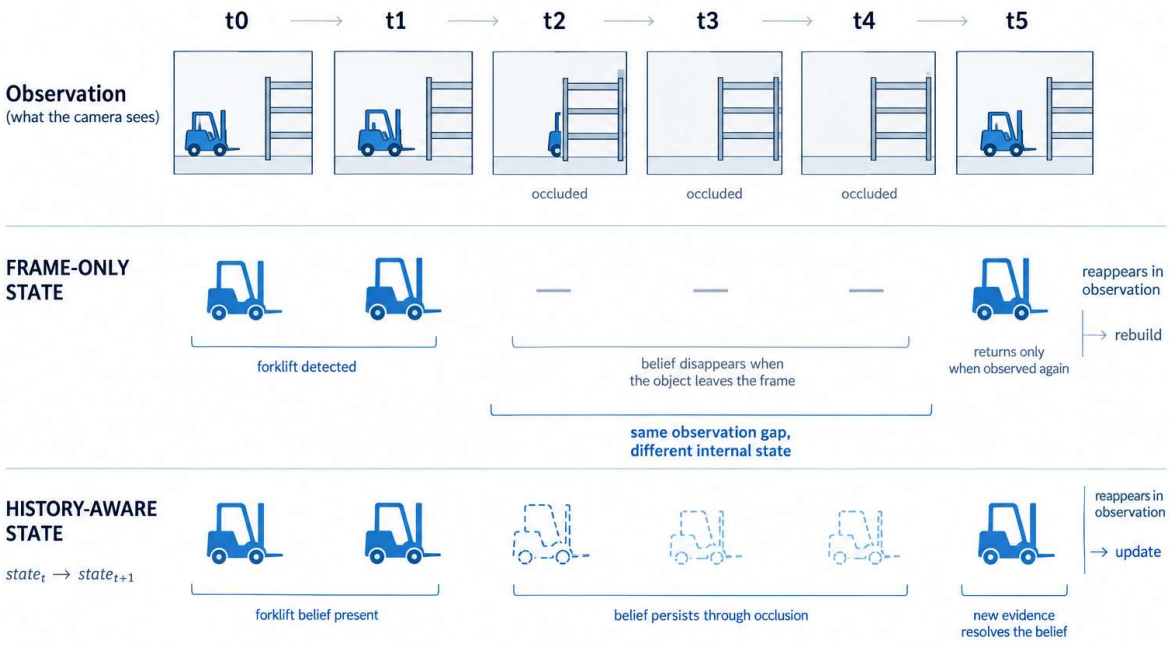


Figure 2.3 A frame-only encoder against a history-aware encoder, tracking the same forklift across five ticks.

A state that survives all of this tells you what's true. It doesn't tell you what happens next. That's the dynamics model's job, and the harder question is whether it can predict the right counterfactual, not merely whether it can predict at all.

2.2. Dynamics and counterfactuals

If we remove the action out of a dynamics model, we're left with a video predictor, which only guesses what happens next without caring about what the agent does. Putting the action in this model makes it action-conditioned, giving it the ability to answer a different question for every candidate the planner wants to try. This distinction is easy to get wrong in practice, because both, a passive and an action-conditioned model, look identical. Both take a state and produce the next state, but only one of them is listening to the action.

Prediction is easier in some environments, because the same state and the same action always lead to the same next state. This is true in cases like chess and most physical simulations, if every variable is controlled. But real life scenarios like our warehouse aren't. It could be a wet patch which nobody logged, decisions of humans, and other random things like the driver feeling a sudden itch making him lose concentration. Using a deterministic model in these scenarios is wrong in a way that hides itself, as it never tells us it might be wrong.

"The model isn't sure" means two different things and calls for different responses.

Aleatoric and epistemic uncertainty

Aleatoric uncertainty comes from randomness in the environment itself, not the model. No amount of additional data removes it, because the world really could go either way. Which direction an unpredictable driver decides to turn is aleatoric. Epistemic uncertainty comes from the model instead. It simply hasn't seen enough situations like this one to know what happens, and more data, or better training, could fix it. Braking on a wet patch the model has never encountered in training is epistemic, right up until it's seen a hundred wet patches, at which point it becomes something the model can predict with confidence.

A good planner must treat both of these differently. Aleatoric uncertainty is a property of the world, so the sane response is risk-aware and robust as it prefers actions that come out fine across a variety of things that might actually happen, rather than just betting everything on the most likely outcome. Epistemic uncertainty is a property of the model's ignorance, and the sane response is closer to caution as it prefers actions that gather more information when it's possible and fall back on a conservative default when it isn't, and treat additional training data as an actual fix. Actually estimating either kind of uncertainty is a training-time concern, and chapter 6 discusses that. So the question that matters here is whether it's the world being random, or the model being out of its depth.

Putting an action into the model's input doesn't automatically mean the model will use it the way we intend it to. Every time our training data showed the robot near the forklift, the robot happened to be braking, because that's a sensible thing for a robot to do near a forklift. A dynamics model trained on that data can learn a shortcut to predict "forklift nearby" whenever it sees "braking" in its input, without ever learning that braking is what caused the robot's own

speed to drop. It picked up a correlation between action and context that happened to exist in the data, instead of the causal effect the action actually has on the future. Feeding that model an action it rarely saw in that context like accelerating near the forklift, there's no guarantee it predicts the right thing because it never learned what accelerating does. It learned what accelerating was usually paired with. So, correlation and causation are different and that difference should be maintained in our model.

That's a deep problem, closer to causal inference than to ordinary supervised learning, and this book isn't solving it here. But its shape matters early, because it explains something that otherwise looks mysterious later on, and that's why the data used to train a dynamics model matters as much as the architecture. It must be diverse and sometimes deliberately random actions during data collection help, because they break the usual pairing between an action and its context, giving the model a better chance of isolating what the action itself causes. That improves coverage and identifiability even if it doesn't guarantee them. Having diverse data makes the causal effect easier to find, but doesn't hand the model the right answer automatically (figure 2.4).

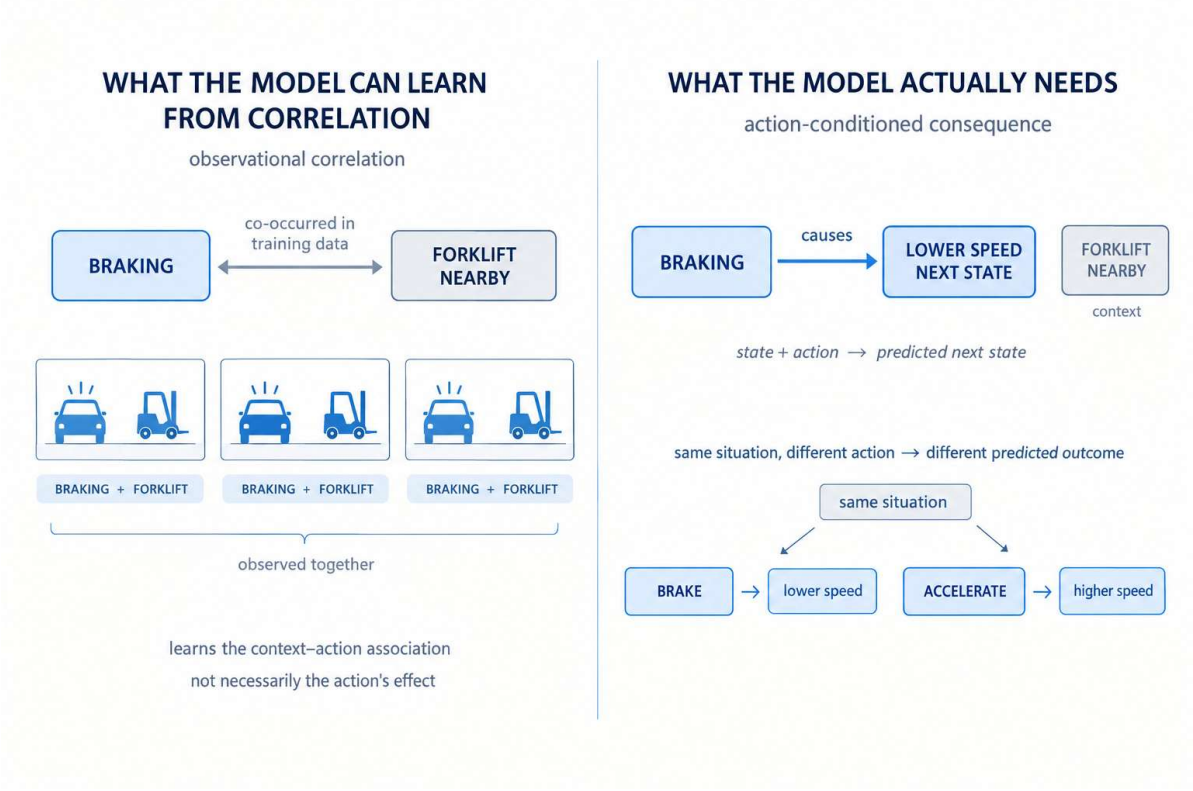


Figure 2.4 Correlation is not the counterfactual.

When we put determinism, uncertainty, and causal grounding together, let's see what actually happens over a long rollout. A one-step model that trusted itself twenty times in a row can end up somewhere it never saw, and "errors add up" becomes an understatement. The dynamics model's own output becomes its next input. So, if step three's prediction is slightly wrong, step four is predicting from a state the model has never actually seen, not from the true state anymore, since training only ever showed it real states paired with real next states. A model asked to extrapolate from an unfamiliar input will get worse, not just proportionally wrong, and that worse output becomes step five's input, and so on. Small errors don't just add, they push the whole rollout progressively further outside the region the model was trained to be good in, the same out-of-distribution problem as the wet patch, except now self-inflicted rather than caused by an unfamiliar surface (figure 2.5).

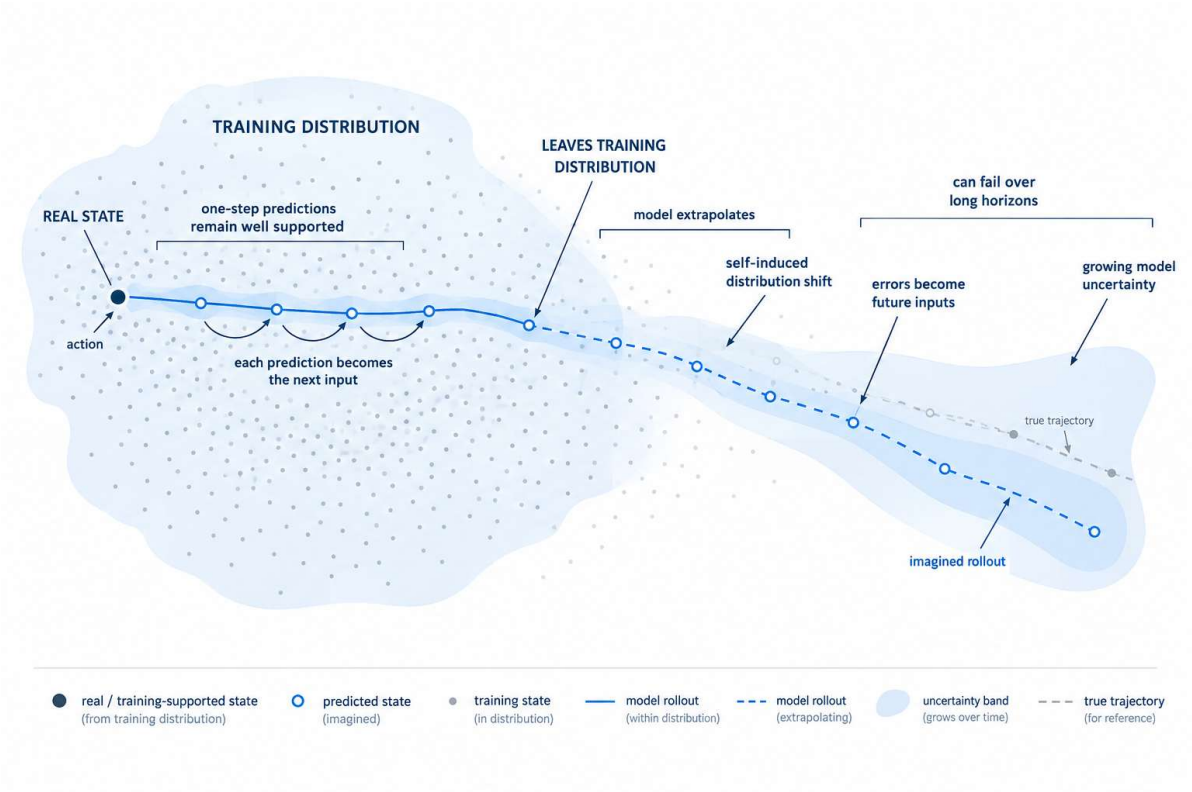


Figure 2.5 How errors compound during imagined rollouts.

There's a simple version of this that shows up before we ever run a rollout which is - how the model was trained. Training almost always shows it real state, real action, real next state, one clean step at a time. Using something trained in this manner asks it to do something it never practiced, predicting from its own previous guess, over and over, twenty times running. This mismatch is called exposure bias. It's one well-studied piece of the picture rather than the

whole picture. In practice, long-horizon degradation is usually several things compounding at once, like ordinary one-step error that's individually small, the distribution shift once the model starts consuming its own output, whatever aleatoric uncertainty the environment always had in itself, and sometimes dynamics that simply weren't learned very stably to begin with. Chapter 6 is where each of these gets its own diagnosis and its own fix.

This is also why the planner keeps only the first action of the best rollout and throws the rest away. An open-loop plan, committed to entirely and executed blind, has no chance to correct for any of this. A closed-loop plan, replanning from a fresh, real observation every tick, only ever has to trust the dynamics model for one step's worth of drift before it gets to check its work against reality again.

Dynamics that can answer the counterfactual question honestly, including admitting when it's guessing, hands the next organ something real to work with. What it does with that something is a different problem.

2.3. Objectives: reward, value, and preferences

A reward is the easy half of what comes next: some number attached to a single transition, how good was it that the robot ended up here, having taken this action from that state. It's local, it's about right now.

Return is what you get when you add up every reward actually collected along one particular trajectory, real or imagined, from now until the end. It's a fact about a specific path, computable only once that path has actually played out. Value is different: an estimate of what return you should expect from a state, without having to imagine every path that could follow from it. Think of return as a single sample and value as the thing you're trying to estimate from samples like it. A planner that could imagine every possible future perfectly would never need value at all, it could just compute the return of each rollout directly and compare. Because rollouts are expensive and horizons are finite, value is the shortcut that lets a rollout stop imagining and start trusting a number instead. We can write value compactly as

$$V(s) \approx \gamma^0 r_0 + \gamma^1 r_1 + \gamma^2 r_2 + \dots$$

the discounted sum of expected future reward, where γ, a number between 0 and 1, is the discount, making a reward ten steps away count for less than one arriving next. That's the only equation this chapter needs. The discount isn't only a mathematical convenience. A γ near 1

tells the system to care almost as much about a reward thirty steps away as one arriving immediately, which is the right instinct for a robot planning a long trip across a warehouse. A γ closer to 0 makes the system almost entirely short-sighted. Neither is universally correct. It depends on how far into the future a bad decision now can actually hurt you (figure 2.6).

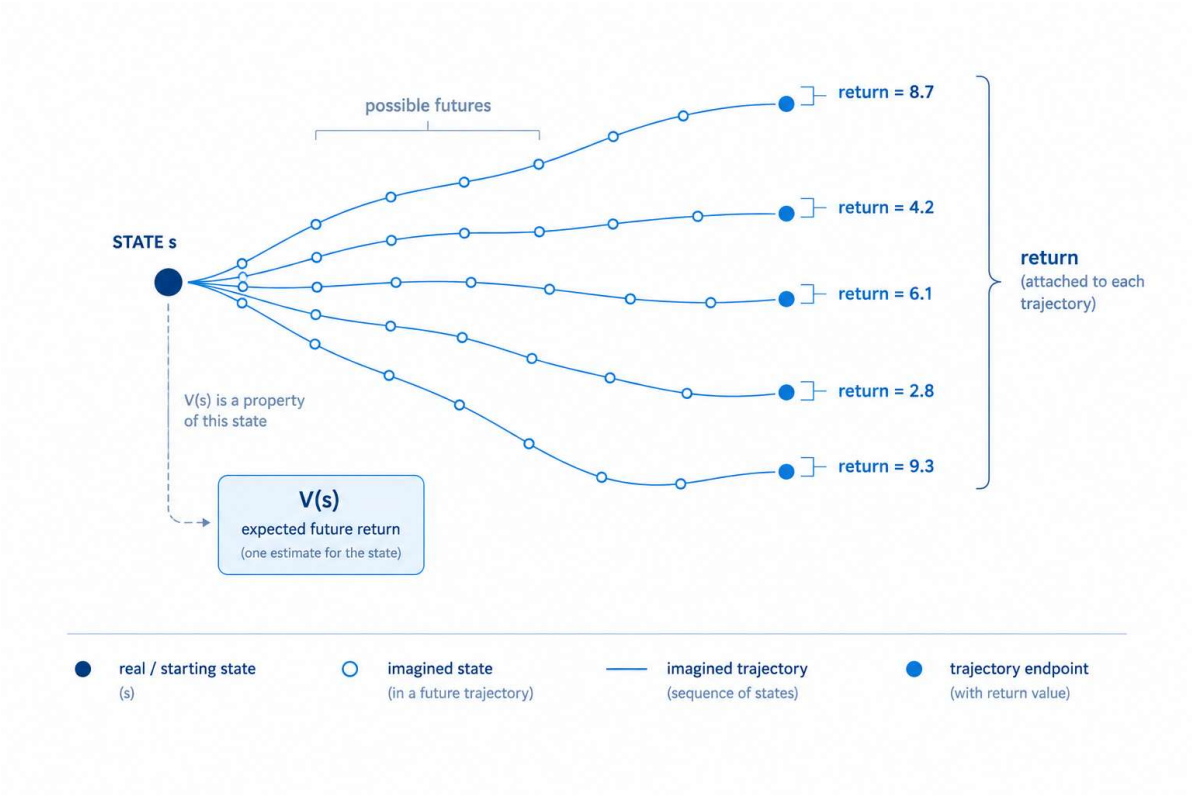


Figure 2.6 Return versus value. From a single state, different imagined trajectories can lead to different future outcomes, with each trajectory producing its own return. Return therefore belongs to one particular path, while value $V(s)$ is an estimate of the expected future return associated with the starting state. Value provides a compact way to summarize possible future outcomes when explicitly imagining every continuation would be too expensive.

A twenty-step rollout that only sums per-step reward and stops is blind past step twenty. If the robot's real goal is thirty steps away, every candidate rollout ends its scoring somewhere in the middle of the warehouse, and none of them can see the payoff that would actually distinguish a good plan from a mediocre one. Gluing a value estimate onto the last imagined state fixes this: sum twenty steps of reward, then add one more number, an estimate of everything that would happen from step twenty onward if the plan kept going the way it's headed. That last addition is usually called bootstrapping the rollout with a value estimate, and it's the difference between a planner that can only see as far as it's willing to simulate, and one with some rough sense, learned from experience, of what lies beyond that. Chapter 7 is where this becomes something you train (figure 2.7).

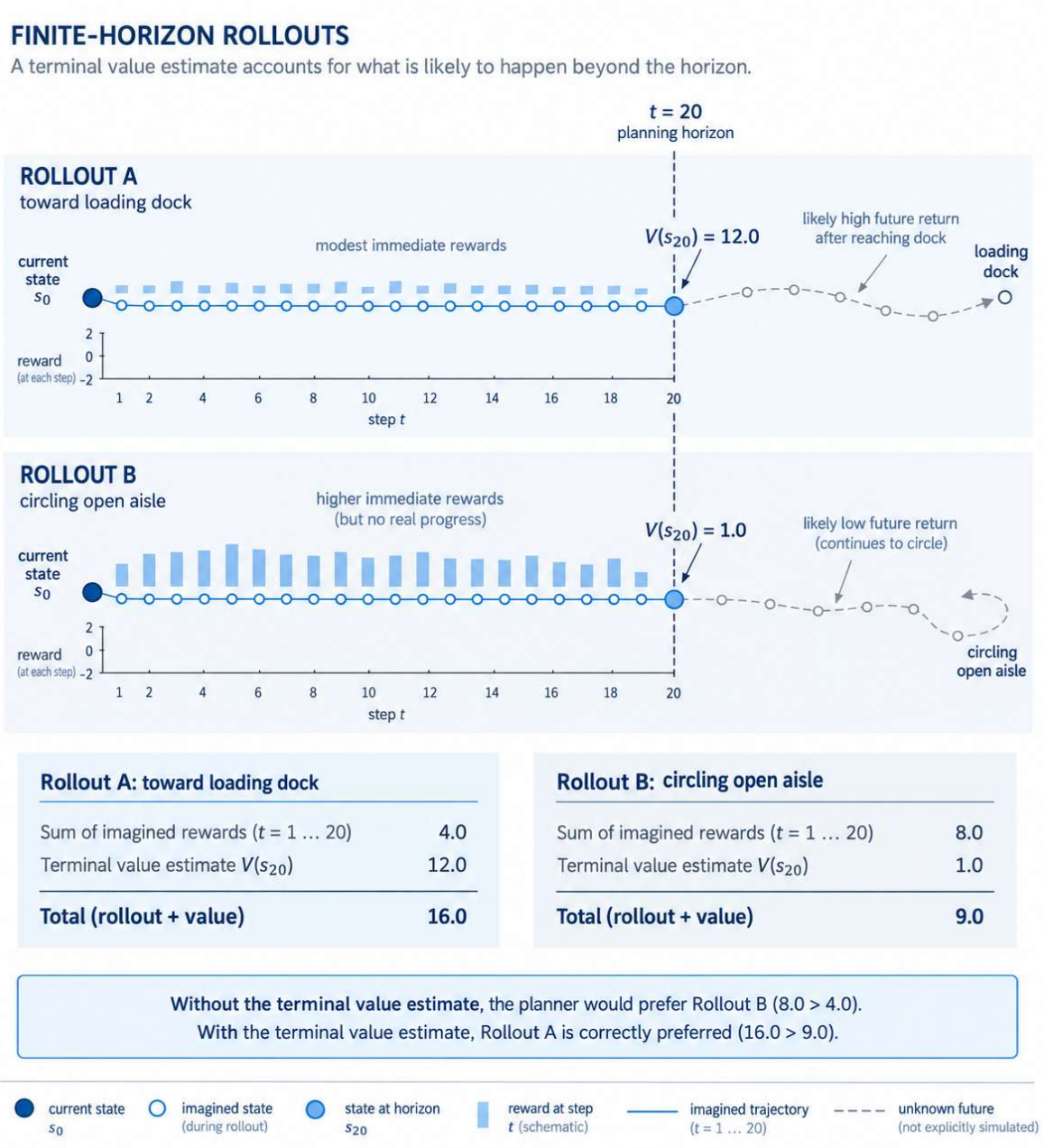


Figure 2.7 Why a finite rollout needs terminal value. Two candidate trajectories are imagined for twenty steps from the same state. The second accumulates slightly more immediate reward during the explicit rollout, but the first ends in a state with much better long-term prospects. At the planning horizon, a terminal value estimate $V(s_{20})$ stands in for the future that is not explicitly simulated. Summing rewards alone would favor the second trajectory; including the terminal value correctly favors the first, allowing a planner to account for consequences beyond its explicit imagination horizon.

A planner can be handed a perfectly good value estimate and still fail, if the number it was told to maximize isn't quite the outcome anyone wanted. Reward it for distance covered per second and it'll drive fast circles in an open aisle, technically maximizing exactly what you asked for

while never approaching the loading dock. That gap, between the proxy you can measure and the objective you actually care about, will follow you into every later chapter that trains a reward or value function. A related problem is credit assignment: when the only reward arrives once, at the end, nothing tells the system which of the twenty actions along the way actually deserved it. Both get their real treatment in chapter 7, once there's a learned value function to hang the discussion on.

> **NOTE** Where the reward actually comes from is its own design discipline, sometimes called reward shaping, and for a lot of real robotics work it takes more careful thought than the dynamics model does.

An objective, once it's honest about what it's actually asking for, still has to get turned into a single decision. That's the next organ.

2.4. Imagination and action selection

Given a state, a dynamics model, and an objective, the most direct way to pick an action is to try several, in imagination, and see which one scores best. Sample a handful of candidate action sequences, roll each one forward through the dynamics model, score the trajectories that come out, and act on whichever scored highest. That's the entire idea. Everything in this section is about the decisions hiding inside how you actually do it.

Look back at Listing 1.1. Inside it are two numbers, 50 and 20, that we breezed past as if they were incidental. They aren't. Listing 2.1 is the same skeleton, annotated with everything sections 2.1 through 2.3 now let us name.

Listing 2.1 Nothing about the shape has changed. What's changed is that every number and every missing line now has a name.

```
PSEUDOCODE
state = empty_state()
loop at each tick:
    obs = environment.observe()
    state = encoder.update(state, obs)     # 2.1: needs to survive occlusion,
not just this frame

    best_score, best_plan = -infinity, none
    repeat 50 times:                         # search budget
        plan = sample_action_sequence(horizon = 20)   # planning horizon
        s = state
        score = 0
        for action in plan:
            s = dynamics.predict(s, action)  # 2.2: trustworthy for a step,
drifts after many
            score = score + reward.estimate(s)
        score = score + value.estimate(s)    # 2.3: without this line, the
rollout is blind past step 20
        if score > best_score:
            best_score, best_plan = score, plan
    environment.act(best_plan[0])            # discard the rest; replan next
tick
```

Constantly replanning and throwing away everything but the first action has an older, more formal name. In control theory it's called receding horizon control, and model-predictive control, the term chapter 1 borrowed from the 1950s, is really just receding horizon control built on a learned dynamics model instead of a hand-derived one.

Four decisions hide in that loop, and none of them are free (table 2.1).

Table 2.1 What each planning decision buys and costs.

Decision	Buys you	Costs you
Longer planning horizon	Sees further into the future before committing	More steps for dynamics error to compound (section 2.2)
Bigger search budget	Better coverage of the action space	Compute that grows with every candidate sampled

Decision	Buys you	Costs you
More frequent replanning	Catches dynamics errors early	More total planning calls over the same stretch of time
A smarter search algorithm	Finds good plans with fewer samples	More complexity to implement and debug

Random shooting, which is exactly what Listing 2.1 does, samples candidates with no strategy and keeps whichever scores best. It's the simplest thing you can write, and most of what it samples is wasted. The cross-entropy method spends the same budget more intelligently, resampling a narrower batch centered on whatever scored best in the previous round, a few times over. Monte Carlo tree search does something similar for discrete action spaces, building an actual tree and spending the next round of search where the early rounds looked promising. If you've read anything about AlphaZero or MuZero, this is the family of ideas behind their move selection. None of these three are architectures. They're strategies for spending a fixed search budget, and chapter 7 is where you'll build one properly.

There's a fundamentally different option: train a policy network ahead of time, on a pile of imagined rollouts, that maps a state directly to an action, no search at decision time. This is amortized planning, because the expensive part happens once, during training, and gets amortized across every future decision (figure 2.8). The trade-off is specific and mechanical, not vague: a policy network only makes sense relative to the reward it was trained against, baked into its weights. Change what you want the robot to optimize for and there's no way to hand it a new scoring function on the fly the way Listing 2.1 can hand reward.estimate a different function and immediately replan. You have to retrain. Search happens at run time or it happens at training time, and whichever you pick, you don't get the other one for free.

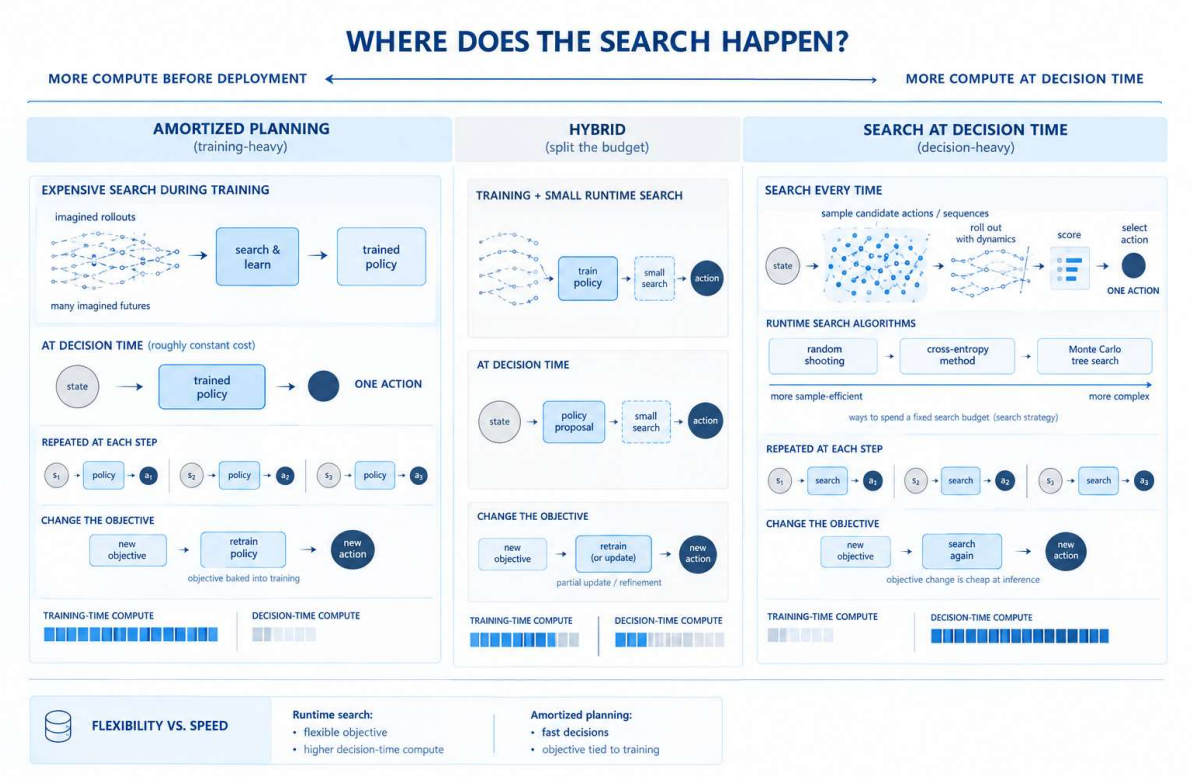


Figure 2.8 Planning as compute allocation. Action selection is a compute-allocation problem: planning/search can happen at decision time, ahead of time during training, or partly in both places. Amortized planning does not eliminate search. It moves much of the search computation from inference into training.

Either way, action selection is a compute-allocation problem wearing a decision's clothes: spend the budget at run time, spend it ahead of time during training, or split it between the two. Nothing about the state, the dynamics, or the objective forces one choice over the other.

2.5. Memory and temporal consistency

Say every organ so far is trustworthy for one tick: state, dynamics, objective, action selection. Run the loop for a few hundred ticks and a different question appears. Does any of it still remember what it learned forty ticks ago, or has it drifted, unnoticed, into believing something that stopped being true a while back?

Drift and staleness

Drift is what happens when a persisting state slowly loses touch with the environment's actual condition, not because of one big mistake, but because small approximation errors in how the state gets updated accumulate over many ticks, the same way small

dynamics errors accumulate over many rollout steps in section 2.2, except here it happens to the belief itself rather than a single prediction. Staleness is the specific case where a particular belief the encoder formed a while ago has since become false, and nothing has arrived to correct it, because there's no negative signal that actively erases old, wrong beliefs, only the absence of anything confirming them.

Say the forklift from section 2.1 actually drove off ten seconds after it went behind the shelf. Nothing about that departure necessarily reaches the robot's sensors, since the shelf blocks the view either way. A state that assumed the forklift was still there, still approaching, is now simply wrong, and stays wrong for as long as nothing shows up to contradict it. Aliasing is two different situations mapped to the same representation right now. Staleness is one situation slowly turning into a different one while the representation doesn't notice, because nothing told it to (figure 2.9).

This sounds like the context-window argument from section 1.4, but it isn't quite the same complaint. Section 1.4's argument was about cost: attention over a growing transcript gets more expensive as the transcript grows, while a bounded, fixed-size state's update cost doesn't grow with tick count at all. This is a different problem entirely. Even holding update cost fixed, a fixed-size vector can still lose track of something it learned forty ticks ago, because whatever training process shaped that vector never taught it to hold information across that span. A bounded state doesn't get memory for free just by being bounded. Something has to actively decide, on every update, how much of what's already there to keep and how much to let a new observation overwrite. This is usually handled with gating, a learned mechanism that makes exactly that call. Older recurrent networks did it crudely and tended to forget things faster than you'd like over long spans. State-space models, which chapter 5 covers properly, were built with this problem in mind, and selective versions like Mamba learn what's worth carrying forward and what's safe to let decay, rather than forgetting at some fixed rate regardless of what actually matters. History is the raw material memory is built from. Memory is what a system does with it on purpose.

There's a simple way to test any of this before a real dynamics model even exists to argue about: hide something from the state on purpose, for a controlled number of ticks, and watch what it does. Does it hold a reasonable belief through the gap? Does it decay that belief sensibly rather than snapping to zero? Does it correctly update once the object reappears, rather than treating the reappearance as something entirely new? This deliberate occlusion

probe turns a fuzzy question, is the memory any good, into something you can actually run and watch happen. The same question shows up again, in a more literal form, once a world model is generating video instead of a compact vector: something leaves the frame, and the honest question is whether the model still knows it exists or simply treats it as gone. Chapter 10 calls this object permanence, and it's the same underlying question wearing pixels instead of a latent vector.

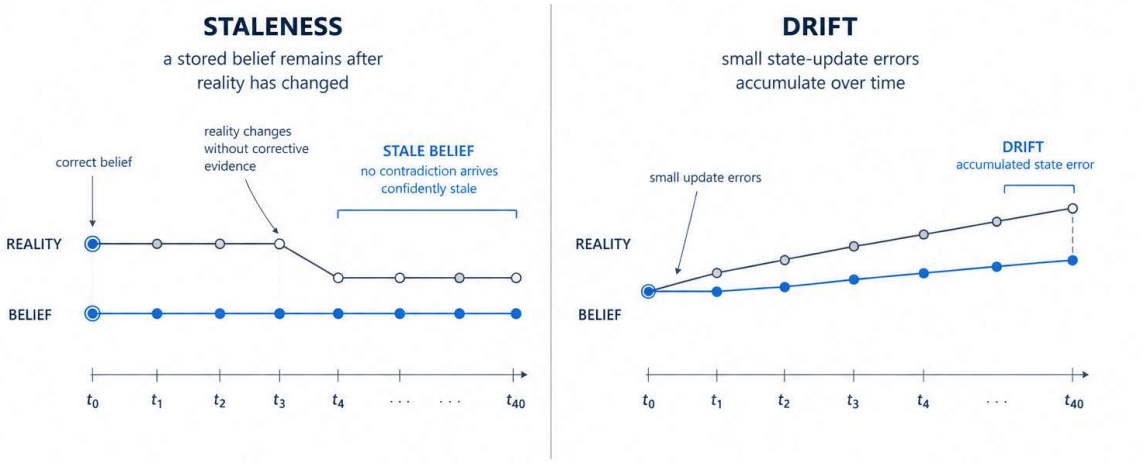


Figure 2.9 Staleness versus drift.

A state, dynamics, an objective, action selection, and now persistence holding all four together over time: that's the full anatomy, working. The next question is how to tell it apart from things that only resemble it.

2.6. What counts as a world model?

A transformer, a recurrent network, a state-space model like Mamba, is a candidate implementation of exactly one organ from this chapter, dynamics. Being good at predicting the next item in a sequence, whether that sequence is words, video frames, or latent states, satisfies "predict how this evolves." It says nothing about the rest.

Next time something claims to be a world model, run three checks. Does it carry an internal representation that persists between calls, or does it get rebuilt from scratch every time. If it's meant to support control, does it actually let a specific action change a specific predicted outcome, the way section 2.2 described, rather than accepting one as an unused input while free-running on whatever it thinks is most likely. And does anything downstream, an explicit planner or a trained policy, actually use its predictions to select an action, or does it just predict and stop. A sequence model can ace next-token or next-frame prediction and still answer no to all three.

Put plainly: every world model's dynamics component is a sequence model of some kind. Not every sequence model is a world model (figure 2.10). That's precisely what was wrong with the coffee cup back in chapter 1. A language model is, among other things, a remarkably capable sequence model over tokens, very good at the narrow contract it was built for. What it's missing is everything sections 2.1, 2.3, and 2.4 were built to check: a representation that persists rather than getting rebuilt from the prompt every time, and something, planner or trained policy, using its predictions to choose between options.

Chapter 1 spent real space on this exact boundary, prediction versus generation, so this isn't a repeat of that argument. What's worth adding now, with the rest of this chapter's vocabulary available, is the boundary test itself. A generative model can pass every visual test you throw at it, sharp, coherent, indistinguishable from something a camera shot, and still fail the same three checks a sequence model fails: no persisting state between calls, no requirement that a specific action changes a specific predicted outcome, and no planner or policy using its output to choose anything. Good generation is a useful debugging tool, chapter 1 already said as much, and several later chapters build systems where generation is the entire product. But a generator earns the name world model only once persistence, action-conditioning, and action selection get built on top of it. Until then, it's a very good painter, which was chapter 1's point about prediction and generation, restated here as a checklist instead of an argument.

There's a third impostor worth naming, and chapter 1 already met it under a different name. A simulator gets its dynamics right by construction: someone wrote the physics equations, set the friction coefficients, and coded the collision rules by hand. Chapter 1 pointed out why this doesn't scale, hand-authoring a faithful simulator of even one warehouse is difficult, doing it for every warehouse, kitchen, and street on the planet is not a research problem, it's a wish. A world model's dynamics are learned from data instead, which trades the simulator's guaranteed correctness for something more useful in practice: dynamics that generalize to

situations nobody hand-coded for, forklift habits included, at the cost of the honesty problem section 2.2 spent its time on. A simulator doesn't need to know when it's guessing. A learned world model does, or it should.

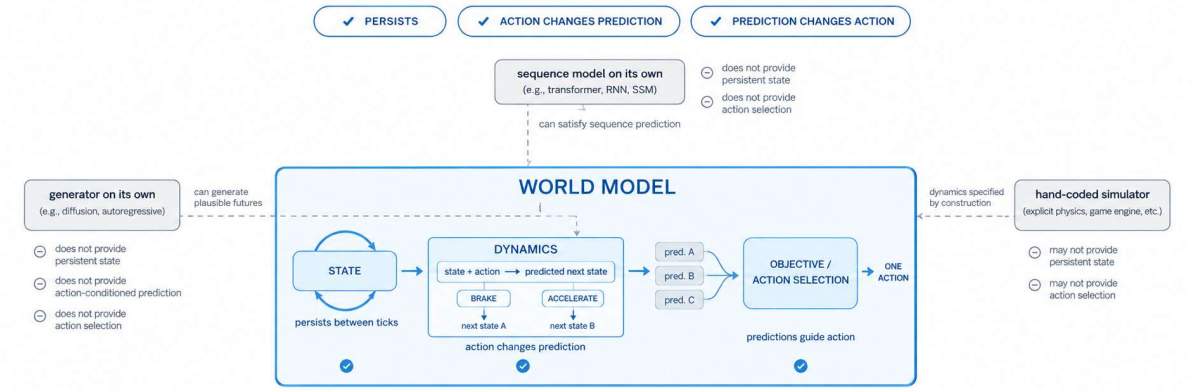


Figure 2.10 What counts as a world model? A sequence model, generator, or simulator can provide some of the predictive machinery associated with a world model without necessarily forming the complete system. The minimum distinction is whether the system maintains a persistent internal representation, whether a specific action changes its predicted outcome when control is the goal, and whether those predictions are actually used to select an action. A complete world model brings these relationships together rather than treating sequence prediction or simulation alone as sufficient.

Every architecture name you'll meet later in this book is a candidate filler for one of the organs this chapter just described, not a new organ. JEPA, in chapter 4, is a way of building the state and the passive half of dynamics without ever reconstructing a pixel. State-space models like Mamba, in chapter 5, are a way of building the persistence section 2.5 kept insisting on, carrying information across ticks through learned gating instead of a growing transcript. Dreamer, in chapter 7, is a way of building action selection as a trained policy instead of a search loop. Diffusion, in chapters 9 and 10, is a way of building dynamics that can also render themselves, generation and prediction sharing one engine. None of them add a part to the anatomy. They're answers to the same questions this chapter just asked, and the rest of the book is the long answer to how well each one answers them.

2.7. Evaluating a world model

A world model earns that name by being useful, not by looking convincing. A model can render gorgeous, coherent footage and be worthless to a planner, or produce vectors nobody would bother visualizing and be exactly what a planner needs. Here's what to actually check:

- **Prediction fidelity.** Not "does this look plausible," but "did it predict the specific things that mattered." A pixel-level objective rewards getting the wallpaper right exactly as much as getting the forklift right, training a model to be excellent at the easy, irrelevant part. Test the task-relevant variables directly, how far off was the predicted forklift position, against a source of ground truth you trust more than the model itself, a simulator or a small hand-labeled set. Comparing predicted pixels against real pixels mostly tells you whether the model is a good painter.
- **Long-horizon consistency.** Drift and staleness are why identity slips over time. This is how you'd catch it slipping. Run a rollout to the full horizon your planner actually uses, not one step, and check whether the forklift is still recognizably the forklift twenty steps in, or blurred into something between a forklift and a shelf, and whether an object teleports or a wheel reverses direction without cause. A model accurate one step at a time can still fail this badly, because one-step accuracy says nothing about whether small errors are compounding into something incoherent.
- **Action grounding.** Feed the same state two clearly different actions, braking hard against flooring it, and check whether the predicted outcomes actually differ in a way that makes physical sense. If the predictions barely change no matter which action you feed in, the model is free-running on whatever it thinks is most likely rather than actually using the action, which makes it worthless for planning even if its unconditional predictions look reasonable in isolation.
- **Planning utility.** The bottom line, and in some sense the only one that actually matters, since the other three mostly explain why this one succeeded or failed. Swap the trained dynamics model out for something deliberately weak, one that predicts no change at all, and see whether the planner's decisions get measurably worse. If they don't, the original model wasn't contributing anything a planner could use, however good its predictions looked in isolation. This kind of ablation, replacing a piece with something intentionally weak and watching what breaks, is one of the more honest tools for finding out whether a component is doing the job you think it's doing.

Those four are the dimensions you test for. What follows is a different kind of list, not a fifth dimension, but the handful of ways all four tend to break together in practice.

Hallucinated dynamics is section 2.2's out-of-distribution problem once it's confidently wrong instead of honestly uncertain. **Temporal drift** is section 2.5's drift, given its proper name. **Poor action grounding** is exactly what the diagnostic above was built to catch.

Evaluation gaps is this whole section's argument in a phrase: the metrics you'd reach for first, visual quality chief among them, are the ones least likely to catch any of the other four. **Long-horizon uncertainty** is section 2.3's blind rollout and section 2.2's compounding error, arriving together at the far end of a plan that's gone further than the model can honestly speak to. Chapter 12 comes back to each of these once you've built enough real architectures to see them firsthand.

None of these five failure modes are abstract worries. They're the specific, recurring ways a world model fails once you actually build one, and now you have names for all of them before writing a single line of a real one.

Seven parts, then, not a list of complaints about chapter 1's sketch. A state, dynamics, an objective, and action selection are the organs. Persistence is what keeps them alive between ticks. Section 2.6 is the boundary that tells you whether something claiming to have these organs actually does. Section 2.7 is the exam you'd give it before trusting it with anything. Whatever chapter 4's JEPA encoder turns out to look like, whatever chapter 9's diffusion-based dynamics model turns out to look like, whatever policy chapter 7 trains, all of it fills in one of these seven parts. None of it replaces the anatomy.

Summary

- A state is only as good as what it lets you predict, and it has to hold that information across time rather than just summarize the current instant. Aliasing is what happens when it doesn't, whether from missing history, a latent space too small, or a belief erased the moment nothing is confirming it.
- Dynamics has to predict the right counterfactual, not just the most likely-looking next state, and it has to recognize its own uncertainty instead of guessing with confidence. Aleatoric uncertainty is the world being random. Epistemic uncertainty is the model being out of its depth.
- Return is what one trajectory actually got. Value is the estimate you use instead of imagining every trajectory, and a rollout without it is blind past whatever horizon it stops looking.
- Action selection, whether it searches at decision time or trained a policy ahead of time, is a compute-allocation problem hiding inside numbers like horizon length, search budget, and replanning frequency.

- Persistent state has to actively decide what to keep on every update, or it drifts and goes stale exactly when it matters most. History is the raw material. Memory is what a system deliberately does with it.
- A sequence model, a generator, and a hand-coded simulator can each satisfy the dynamics organ alone and still not be a world model. What's missing is persistence, real action-conditioning, and something using the predictions to choose.
- Visual quality is one of the least informative signals available for judging whether any of it actually works. Prediction fidelity, long-horizon consistency, action grounding, and planning utility each test something visual quality was never built to see.

Last updated 2026-09-10 07:13:17 +0530